K A S U

Security Audit Report

RWA Private Credit Lending Platform

Auditor: Claude by Anthropic (AI-Assisted Review)

Date: April 9, 2026

Commit: `b18e98716c3720e821699f081520484648d6d729`

Branch: release-candidate

Solidity Version: 0.8.23

Scope: 40 contracts + 2 vendor · ~9,728 LoC · Full + Lite deployments

AI-Assisted Security Review — Not a Replacement for Professional Audit

kasu.finance · Docs · Twitter/X · Discord · Telegram · Medium

K A S U
Security Audit Report — April 2026

IMPORTANT NOTICE

Disclaimer

Disclaimer

This security audit was performed by Claude (Anthropic), an AI assistant. While AI-assisted reviews can identify many classes of vulnerabilities and provide valuable security insights, this report has inherent limitations:

- AI models may miss subtle vulnerabilities that require deep domain expertise or novel attack vectors.
- This review is based on static analysis of source code. No dynamic testing, formal verification, or fuzzing was performed.
- Cross-contract interactions and deployment-specific configurations may not be fully captured.
- This report should be treated as a supplement to, not a replacement for, a professional security audit by an established firm.
  The Kasu team is strongly encouraged to engage a professional audit firm (e.g., Trail of Bits, OpenZeppelin, Cyfrin, Spearbit) for a comprehensive review before mainnet launch or significant upgrades.

All findings reflect the auditor's assessment at the time of review and are subject to the constraints above.

NAVIGATION

Table of Contents

Executive Summary

Scope & Methodology

Assumptions & Trust Model

Findings Summary

Resolved Findings

Medium Severity Findings (Remaining)

Low Severity Findings

Informational Findings

Security Checklist

Go / No-Go Recommendation

Appendix A: Contract Inventory

Appendix B: Comparison with Prior Audits

K A S U
Security Audit Report — April 2026

S E C T I O N 1

Executive Summary

Overall Risk Assessment: MEDIUM

The Kasu protocol demonstrates strong security engineering with comprehensive role-based access control, proper use of OpenZeppelin v5 libraries, epoch-based design that eliminates most MEV vectors, and defense-in-depth patterns throughout. This audit was conducted using three complementary methodologies (Feynman deep logic analysis, State Inconsistency structural analysis, and Nemesis iterative fusion), identifying 1 high, 8 medium, 6 low, and 7 informational findings.

Five findings have been resolved during this audit cycle (H-01, M-01, M-02, M-05, M-07). All fixes have been verified as correct and complete with dedicated test coverage. Four medium-severity findings remain unresolved and are tracked for the next audit cycle.

0	0	4	6	7
CRITICAL	HIGH	MEDIUM	LOW	INFORMATIONAL

5 additional findings (1 High, 4 Medium) were identified and **resolved** during this audit cycle.

Top Risks

Admin privilege risks without guardrails — `emitUserLoyaltyRewardBatch` allows arbitrary reward inflation. All config changes take immediate effect without timelock (M-03).
Stale KSU epoch price inflates loyalty levels — If `updateKsuEpochTokenPrice()` is not called each epoch, users get inflated loyalty levels and unfair clearing priority (M-04).
Incomplete loss token minting blocks clearing — Reporting a loss with `doMintLossTokens = false` can block clearing step 4 until all loss tokens are minted (M-06).
Withdrawal pro-rata truncation permanently delays small positions — Integer division truncation to 0 causes small withdrawal requests to be stuck indefinitely (M-08).

K A S U
Security Audit Report — April 2026

S E C T I O N 2

Scope & Methodology

Scope

All Solidity implementation contracts in the `src/` directory and vendored dependencies in `vendor/`, covering all deployment modes (Base Full, Plume Lite, XDC Lite).

Category	Files	Lines of Code
Core — Lending Pool	10 files	~4,560
Core — Clearing	5 files	~1,533
Core — Config & Management	8 files	~1,850
Core — Swap	2 files	~279
Locking System	4 files	~656
Token & Access Control	5 files	~477
Lite Variants	5 files	~246
Shared Utilities	3 files	~127
Vendor (Nexera)	2 files	~293
Total	**44**	**~10,021**

Methodology

This audit employed three complementary methodologies executed in parallel, with a final iterative fusion pass:

1. Feynman Deep Logic Audit — First-principles reasoning applied to every function. Questions every line's purpose, ordering, guards, and implicit assumptions. Finds business logic bugs that pattern-matching misses. Applied across 3 parallel workstreams covering all contracts.

2. State Inconsistency Audit — Structural invariant analysis mapping all coupled state pairs, building mutation matrices, and cross-checking every mutation path for missing state synchronization. 15 coupled state pairs mapped, ~50 mutation paths analyzed.

3. Nemesis Fusion Pass — Iterative cross-pollination of Feynman and State Inconsistency findings. Tests intersection hypotheses where bugs from different methodologies compound. 3 intersection hypotheses investigated.

K A S U
Security Audit Report — April 2026

S E C T I O N 3

Assumptions & Trust Model

Privileged Roles

Role	Holder	Capabilities
`DEFAULT_ADMIN_ROLE` / `ROLE_KASU_ADMIN`	Kasu Multisig	Pause system, configure fees/thresholds, manage roles, recover tokens, block users
`ROLE_POOL_ADMIN`	Pool Admin Multisig	Create pools, configure pool parameters, manage pool roles
`ROLE_POOL_MANAGER`	Pool Manager Multisig	Update draw amounts, repay funds, report losses
`ROLE_POOL_CLEARING_MANAGER`	Pool Admin Multisig	Execute clearing process
`ROLE_PROTOCOL_FEE_CLAIMER`	Configured address	Claim accumulated protocol fees

Upgradeability

Transparent Proxy (per-proxy ProxyAdmin, OZ v5) for core singletons. Beacon Proxy for pool instances (LendingPool, PendingPool, LendingPoolTranche). Each proxy has its own dedicated ProxyAdmin contract. All ProxyAdmin and Beacon ownership is held by the Kasu multisig.

External Dependencies

USDC — Underlying asset for all lending pools. Trusted as non-reentrant ERC20.
Compilot (NexeraID) — KYC/KYB signature gating. Off-chain signer controlled by third party.
OpenZeppelin Contracts v5.0.2 — Base contracts for access control, tokens, proxies.

Trust Model

Admin multisigs are trusted. Inter-contract calls between Kasu system contracts are trusted. End users interact only through LendingPoolManager. KYC signatures are validated on-chain via the vendored NexeraID verifier.

K A S U
Security Audit Report — April 2026

S E C T I O N 4

Findings Summary

ID	Severity	Title	Status
H-01	RESOLVED	Stale `_isUserPartOfLendingPool` mapping corrupts user re-enrollment	Fixed
M-01	RESOLVED	Fee emission permanently lost when no eligible rKSU holders	Fixed
M-02	RESOLVED	`recoverERC20` can drain KSU reward pool	Fixed
M-03	MEDIUM	`emitUserLoyaltyRewardBatch` allows arbitrary reward inflation	Open
M-04	MEDIUM	Stale KSU epoch price inflates loyalty levels	Open
M-05	RESOLVED	Loss repayment no upper bound	Fixed
M-06	MEDIUM	Incomplete loss token minting blocks clearing step 4	Open
M-07	RESOLVED	Balance-based swap accounting includes residual tokens	Fixed
M-08	MEDIUM	Withdrawal pro-rata truncation permanently delays small positions	Open
L-01–L-06	LOW	6 low-severity findings (see section 7)	Open
I-01–I-07	INFO	7 informational findings (see section 8)	Open

K A S U
Security Audit Report — April 2026

S E C T I O N 5

Resolved Findings

The following findings were identified during this audit and resolved before report finalization. All fixes have been verified as correct and complete with dedicated test coverage (195 tests passing).

H-01 RESOLVED Stale `_isUserPartOfLendingPool` mapping corrupts user re-enrollment and clearing priority

CONTRACT

`UserManager.sol` — `_removeLendingPoolFromUser()`

IMPACT

When a lending pool was removed from a user's `_userLendingPools` array via swap-and-pop, the `_isUserPartOfLendingPool[pool][user]` mapping was never reset to `false`. Re-depositing into the same pool skipped re-adding it, leaving the user's pool list empty. Users with rKSU got `type(uint256).max` loyalty ratio (unfair clearing priority); users without rKSU got zero loyalty rewards.

FIX APPLIED

Added `_isUserPartOfLendingPool[_userLendingPools[user][idx]][user] = false;` before the swap-and-pop. The mapping is read before the swap overwrites the element. Pattern matches existing `_removeUserFromAllUsers` which follows the same set-false-then-swap approach.

VERIFICATION

2 dedicated tests added: `test_H01_reDepositAfterCleanup_poolMustBeReTracked` and `test_H01_loyaltyLevelCorrectAfterReDeposit`. Both pass. Full suite: 195/195.

M-01 RESOLVED Fee emission permanently lost when no eligible rKSU holders

CONTRACT

`KSULocking.sol` — `emitFees()` / `FeeManager.sol` — `emitFees()`

IMPACT

When `eligibleRKSUForFees == 0`, USDC fees were transferred to KSULocking but `_accumulatedRewardsPerShare` was not updated, permanently stranding the fees.

FIX APPLIED

Two-layer defense: (1) `KSULocking.emitFees()` reverts with `NoEligibleFeeRecipients()` when no eligible holders. (2) `FeeManager.emitFees()` checks `eligibleRKSUForFees > 0` before calling KSULocking; when false, redirects all fees to protocol fees. No dangling approval left when skipping.

VERIFICATION

2 dedicated tests added. Base mainnet deployment confirmed not affected (FeeManager already had the guard; no stranded USDC). Full suite: 195/195.

M-02 RESOLVED `recoverERC20` can drain KSU reward pool beyond unclaimed balance

CONTRACT

`UserLoyaltyRewards.sol`

FIX APPLIED

Removed `recoverERC20` function entirely from `UserLoyaltyRewards.sol`, `UserLoyaltyRewardsLite.sol`, and `IUserLoyaltyRewards.sol`. No references remain in `src/`.

VERIFICATION

1 dedicated test confirms function no longer exists via low-level call. Full suite: 195/195.

M-05 RESOLVED Loss repayment has no upper bound; `recoveredAmount` can exceed `lossAmount`

CONTRACT

`LendingPoolTrancheLoss.sol` — `repayLoss()`

FIX APPLIED

Added check: `if (recoveredAmount + amount > lossAmount) revert RecoveryExceedsLoss(lossId)` before incrementing. Uses `>` (not `≥`) so exact-amount repayment succeeds. No overflow concern — Solidity 0.8.23 has built-in overflow checks.

VERIFICATION

1 dedicated test confirms exact repayment succeeds and 1 additional wei reverts. Full suite: 195/195.

M-07 RESOLVED Balance-based swap accounting includes residual tokens

CONTRACT

`DepositSwap.sol` — `_transferAndSwap()`

FIX APPLIED

Changed from absolute `balanceOf(this)` to delta-based accounting: capture `balanceBefore` before swap, compute `outTokenAmount = balanceOf(this) - balanceBefore` after. If the swap somehow reduces balance, Solidity 0.8.23 reverts on underflow (correct behavior).

VERIFICATION

Fix verified via code trace. Full suite: 195/195.

K A S U
Security Audit Report — April 2026

SECTION 6

Medium Severity Findings (Remaining)

M-03 MEDIUM `emitUserLoyaltyRewardBatch` allows arbitrary reward inflation

CONTRACT

`UserLoyaltyRewards.sol` — `emitUserLoyaltyRewardBatch()`

IMPACT

Admin can call with arbitrary parameters (any user, any `amountDeposited`, any `ksuTokenPrice`) without on-chain validation against actual state. A compromised admin multisig could inflate rewards and drain all KSU from the contract.

ROOT CAUSE

Batch emission function trusts admin input without on-chain state validation.

RECOMMENDED FIX

Add a maximum reward cap per call. Ensure admin is a well-configured multisig with timelock. Document as intentional admin privilege.

M-04 MEDIUM Stale KSU epoch price inflates loyalty levels when `updateKsuEpochTokenPrice` not called

CONTRACT

`SystemVariables.sol` / `UserManager.sol`

IMPACT

`updateKsuEpochTokenPrice()` must be called each epoch by an external caller. If not called, `ksuEpochTokenPrice` remains at its last-updated value. A stale high price inflates loyalty levels, giving unfair clearing priority and higher KSU rewards.

ROOT CAUSE

No enforcement that `updateKsuEpochTokenPrice()` is called every epoch. No automated mechanism.

RECOMMENDED FIX

Call `updateKsuEpochTokenPrice()` at the start of `batchCalculateUserLoyaltyLevels`, or use the live price directly.

M-06 MEDIUM Incomplete loss token minting blocks clearing step 4

CONTRACT

`LendingPoolTrancheLoss.sol` / `LendingPoolTranche.sol`

IMPACT

When a loss is reported with `doMintLossTokens = false`, the `notPendingLossMint` modifier blocks all `deposit()` and `redeem()` on the tranche until batch minting completes. Clearing step 4 calls these functions, causing clearing to revert with `LossMintingInProgress`.

ROOT CAUSE

`ClearingCoordinator.doClearing` does not check for pending loss mints before attempting step 4.

RECOMMENDED FIX

Require `doMintLossTokens = true` for production usage. Document operational guidelines for the parameter.

M-08 MEDIUM Withdrawal pro-rata truncation to zero permanently delays small positions

CONTRACT

`AcceptedRequestsExecution.sol`

IMPACT

When computing accepted withdrawal shares pro-rata, small positions truncate to 0 due to integer division. The withdrawal NFT survives with full shares amount, and the user gets 0 shares accepted. This repeats every epoch — the request is effectively stuck permanently.

ROOT CAUSE

Integer division truncation with no minimum acceptance threshold or dust-clearing mechanism.

RECOMMENDED FIX

Set a minimum accepted amount, round up for smallest positions, or allow users to cancel and re-submit.

K A S U
Security Audit Report — April 2026

S E C T I O N 7

Low Severity Findings

ID	Title	Contract
L-01	Max deposit check only applies to pending deposits, not total user position. Repeated epoch deposits can exceed `maxDepositAmount`.	PendingPool
L-02	Deposit round-up (`Math.ceilDiv`) can aggregate dust-level over-acceptance across priority buckets.	AcceptedRequestsExecution
L-03	`batchSize=0` accepted silently as no-op in clearing steps 2 and 4, wasting gas without warning.	PendingRequestsPriorityCalc
L-04	`Swapper._approveMax` uses `safeIncreaseAllowance(type(uint256).max)`. If prior allowance not cleared, next call overflows and reverts.	Swapper
L-05	`_setLoyaltyThresholds` uses strict `>` comparison, allowing duplicate adjacent threshold values that collapse loyalty levels.	SystemVariables
L-06	`_totalUserTranchePendingDepositAmount` tracks pending deposits per user but does not account for existing tranche shares when checking `maxDepositAmount`.	PendingPool

S E C T I O N 8

Informational Findings

ID	Title	Contract
I-01	Zero-amount protocol fee claim succeeds with misleading `ProtocolFeesClaimed` event.	FeeManager
I-02	Asymmetric grant/revoke access: Factory can grant but not revoke pool roles; Kasu Admin can revoke but not grant (by design).	KasuController

ID	Title	Contract
I-03	`emitUserLoyaltyReward` access check occurs after early return, allowing unauthorized callers when `doEmitRewards` is false (no state change).	UserLoyaltyRewards
I-04	Integer division rounding on partial KSU unlocks causes rKSU dust accumulation.	KSULocking
I-05	ERC4626 inflation attack mitigated by 12-decimal offset — economically infeasible.	LendingPoolTranche
I-06	`feesOwedAmount -= overpaidFees` could theoretically underflow in extreme single-depositor edge case.	LendingPool
I-07	Non-transferable tokens (PendingPool NFTs, tranche shares, loss tokens) correctly enforced via function overrides.	Various

K A S U
Security Audit Report — April 2026

S E C T I O N 9

Security Checklist

Category	Status	Notes
Access Control	**PASS**	Comprehensive role-based access via KasuController. All state-changing functions properly guarded by modifiers. Pool-scoped roles via `KasuAccessControllable`. Stale mapping issue (H-01) has been resolved.
Reentrancy	**PASS**	No explicit reentrancy guards, but underlying asset is USDC (no transfer hooks). All sensitive functions access-controlled to trusted system contracts. SafeERC20 used throughout. CEI pattern generally followed.
Arithmetic	**PASS**	Solidity 0.8.23 provides checked arithmetic. `unchecked` blocks used judiciously for loop counters. Interest rates use `INTEREST_RATE_FULL_PERCENT = 1e18`. ERC4626 offset of 12 decimals prevents inflation.
External Calls	**PASS**	All token transfers via SafeERC20. No low-level `call` to untrusted addresses. Swapper uses `Address.functionCall` to allowlisted exchanges only. Swap accounting now uses delta-based pattern (M-07 fix).
Upgradability	**PASS**	TransparentProxy with per-proxy ProxyAdmin (OZ v5). BeaconProxy for pool instances. `_disableInitializers()` called in constructors. `initializer` modifier on all init functions.
Oracle Dependencies	**PASS w/ NOTE**	No DeFi price oracles for core lending. KSU price is admin-set (only affects loyalty, not accounting). `updateKsuEpochTokenPrice` is permissionless but once-per-epoch and can go stale (M-04).
MEV Exposure	**PASS**	Epoch-based clearing eliminates real-time MEV. Deposits/withdrawals queued at epoch boundaries. Clearing by permissioned roles. Swap deposits have `minAmountOut` slippage protection.

Category	Status	Notes
ERC Compliance	**PASS**	ERC20, ERC721, ERC4626, ERC1155 all follow OpenZeppelin implementations. Non-standard behavior (non-transferable tokens) is intentional.
State Consistency	**PASS**	15 coupled state pairs mapped; 1 desynchronization found and fixed (H-01). Clearing pipeline and fee distribution structurally sound.

K A S U
Security Audit Report — April 2026

SECTION 10

Go / No-Go Recommendation

Recommendation: CONDITIONAL GO

The Kasu protocol has a well-engineered architecture with strong defense-in-depth patterns. The permissioned, epoch-based design eliminates most common DeFi attack vectors. No critical or high-severity vulnerabilities remain. Five findings (1 High, 4 Medium) were resolved during this audit cycle with verified fixes. However, the following conditions should be met:

Should Fix (before next audit cycle)

ID	Finding	Rationale
M-03	Batch reward inflation	Admin privilege risk. Consider max reward cap or timelock.
M-04	Stale KSU epoch price	Operational risk. Consider automating price update in clearing flow.
M-06	Loss minting blocks clearing	Operational DoS risk. Document guidelines for `doMintLossTokens`.
M-08	Withdrawal truncation	Affects small positions. Consider dust-clearing mechanism.

Positive Observations

ERC4626 inflation well-mitigated — 12-decimal offset makes inflation attacks economically infeasible.
Comprehensive verification in clearing — `_verifyResult` in AcceptedRequestsCalculation provides thorough post-computation invariant checking.
Well-designed batch processing — Steps 1, 2, and 4 support batching to prevent gas-limit DoS.
Proper use of OpenZeppelin v5 — Per-proxy ProxyAdmin, SafeERC20, AccessControl, ERC4626 with virtual assets.
Epoch-based design eliminates MEV — No real-time pricing, no flash loan vectors, no sandwich attacks.

Clean Lite variant architecture — Lite contracts properly stub out token-dependent functionality while preserving KYC and core deposit flows.
Strong coupled state consistency — Of 15 mapped coupled state pairs, the only desynchronization (H-01) has been fixed.
Non-transferable tokens — PendingPool NFTs, tranche shares, and loss tokens are all correctly restricted.
Responsive remediation — 5 findings resolved during audit cycle with verified fixes and dedicated test coverage.

K A S U
Security Audit Report — April 2026

S E C T I O N 1 1

Appendix A: Contract Inventory

Core — Lending Pool

Contract	LoC	Description
LendingPool.sol	1,229	Pool ledger, ERC20 token, interest/loss/draw accounting
LendingPoolManager.sol	836	Entry point for all user and admin interactions
PendingPool.sol	823	ERC721 pending request management, extends ClearingSteps
FixedTermDeposit.sol	817	Fixed-rate locked deposits with clearing integration
LendingPoolTrancheLoss.sol	298	ERC1155 impairment receipts, loss registration and repayment
LendingPoolTranche.sol	270	ERC4626 vault for tranche shares
LendingPoolFactory.sol	182	BeaconProxy-based pool deployment
LendingPoolStoppable.sol	~40	Pool stopping mechanism
LendingPoolHelpers.sol	62	Shared helper functions
UserRequestIds.sol	~50	Compact ID encoding library

Core — Clearing

Contract	LoC	Description
AcceptedRequestsCalculation.sol	459	Pure calculation of accepted deposits/withdrawals
ClearingCoordinator.sol	334	Orchestrates 5-step clearing pipeline
PendingRequestsPriorityCalculation.sol	287	Priority calculation by loyalty level
AcceptedRequestsExecution.sol	255	Pro-rata execution of accepted requests
ClearingSteps.sol	197	Abstract contract bridging clearing to PendingPool

Core — Configuration & Management

Contract	LoC	Description
UserManager.sol	543	Loyalty level calculation, user tracking
SystemVariables.sol	470	Global system config (epochs, fees, KSU price)
UserLoyaltyRewards.sol	232	KSU rewards based on liquidity and loyalty
KasuPoolExternalTVL.sol	162	ERC1155 for off-chain TVL tracking
DepositSwap.sol	140	Swap-and-deposit functionality
Swapper.sol	139	DEX swap execution with allowlist
KasuAllowList.sol	132	KYC verification via NexeraID signatures
FeeManager.sol	108	Ecosystem/protocol fee splitting

Locking, Token & Access Control

Contract	LoC	Description
KSULocking.sol	478	KSU token locking with fee distribution
KSU.sol	50	ERC20 token with permit and burn
KasuController.sol	139	Role management, system pause
KasuAccessControllable.sol	101	Base contract for pool-scoped access
Roles.sol	25	Role constant definitions
rKSU.sol	43	Non-transferable receipt token
KSULockBonus.sol	37	Bonus KSU distribution for lockers

Lite Variants & Vendor

Contract	LoC	Description
KSULockingLite.sol	102	Stub locking for token-free deployments
UserManagerLite.sol	60	Disabled loyalty calculations
ProtocolFeeManagerLite.sol	37	Protocol-only fee handling

Contract	LoC	Description
UserLoyaltyRewardsLite.sol	32	Disabled reward emissions
KsuPriceLite.sol	14	Returns 0 for KSU price
vendor/nexera/BaseTxAuthDataVerifier.sol	215	Core KYC signature verification
vendor/nexera/TxAuthDataVerifierUpgradeable.sol	78	Upgradeable wrapper for KYC verifier

K A S U
Security Audit Report — April 2026

S E C T I O N 1 2

Appendix B: Comparison with Prior Audits

February 2026 Audit Findings — Current Status

Feb 2026 ID	Title	Apr 2026 Status
H-01	FTD batch array corruption	Not reproduced (false positive — reverse iteration is correct)
H-02	Stale nextIndexToProcess in FTD	Not reproduced (related to H-01)
M-01	UserManager stale mapping	**Confirmed & Fixed** (elevated to H-01, now resolved)
M-02	recoverERC20 drain	**Confirmed & Fixed** (function removed)
M-03	Batch reward inflation	Confirmed (M-03, open)
M-04	Unbounded _allUsers	Acknowledged (operational concern)
M-05	block.number KYC expiry	Acknowledged (vendored dependency)
M-06	Balance-based swap	**Confirmed & Fixed** (M-07, now resolved)
M-07	Loss minting blocks ops	Confirmed (M-06, open)
M-08	FTD fallback wrong deposit	Acknowledged
M-09	Fee loss no holders	**Confirmed & Fixed** (M-01, now resolved)
M-10	ERC1155Burnable bypass	Acknowledged (not directly exploitable)
M-11	Swapper max approval	Downgraded to L-04
M-12	Config immediate effect	Acknowledged

New Findings in April 2026 Audit

ID	Severity	Title	Status
M-04	MEDIUM	Stale KSU epoch price inflates loyalty levels	Open

ID	Severity	Title	Status
M-05	MEDIUM	Loss repayment has no upper bound	**Fixed**
M-08	MEDIUM	Withdrawal pro-rata truncation	Open
L-01	LOW	Max deposit check only on pending	Open
L-02	LOW	Deposit round-up dust	Open
L-06	LOW	Pending deposit vs total position tracking	Open

K A S U

Redefining Real-World Yield — RWA Private Credit

kasu.finance · Documentation · Twitter/X · Discord · Telegram · Medium

hello@kasu.finance

End of Report — Generated by Claude (Anthropic) — April 9, 2026
Commit: b18e98716c3720e821699f081520484648d6d729 · Branch: release-candidate
Methodology: Feynman + State Inconsistency + Nemesis Fusion
Test Suite: 195 tests passing · 5 findings resolved during audit